MINIGAME HOST CONTRACT — v1 (FROZEN)

Purpose: this interface is fixed so all three of us can build in parallel without waiting on each other. Arinze writes games against it. Luca writes the host against it. Neither needs the other's code to exist.

Do not change v1 unilaterally. If a game genuinely can't be expressed here, raise it and we version to v1.1 together.

It generalizes the pattern the Ch 5 decrypt already uses (`state.flags.__minigame__` → Terminal.jsx), so nothing existing has to be rewritten — only extended.

---

1. A minigame is a pure component

Every game — all ten — is one file exporting one component with exactly this signature:

```
export default function MyGame({ config, timeLeft, onResolve }) { ... }
```

That is the entire surface. A game never imports the engine, never touches game state, never advances a beat, never writes a flag, never runs its own countdown.

Props in

Prop	Type	Meaning
`config`	object	`{ id, label, timeLimitSeconds, params }` — `params` is free-form per game (grid size, marker count, difficulty…)
`timeLeft`	number	Seconds remaining. **The host owns the timer.** Render it however you like; you never manage it.
`onResolve`	function	Call exactly once when the game ends.

The one call out

```
onResolve({
  outcome: 'success' | 'failure',   // required
  quality: 'high' | 'low',          // optional — for degraded-success games
  meta:    { ... },                 // optional — anything worth logging
});
```

Call it once. The host ignores repeat calls. If the clock hits zero the host calls `failure` for you and unmounts — you don't have to handle timeout.

---

2. What the host guarantees

The host (Terminal / Lens / Feeds shell) promises to:

1. Mount the right game when `flags.__minigame__` appears, keyed by `config.id`.
2. Run the countdown and pass `timeLeft` down.

3. Auto-resolve `failure` on timeout.
4. On resolve: write the quality flag, clear `__minigame__`, wait ~1.8 s on the result screen, then `engine.advanceBeat(on_success | on_failure)`.
5. Render the idle screen when there's no active minigame.
6. Handle save/reload mid-game (worst case: the game restarts — acceptable).

Games get **none** of this responsibility.

---

3. Registry — the only shared file

```
// src/screens/minigames/registry.js
import MarkerWatch      from './MarkerWatch';
import FrequencyTrace   from './FrequencyTrace';

export const MINIGAMES = {
  ch6_marker_watch:   { app: 'lens',     component: MarkerWatch },
  ch2_frequency_trace:{ app: 'terminal', component: FrequencyTrace },
};
```

Adding a game = **one import line + one registry line.** That's the entire integration step, and it's the only file two people might touch at once — a one-line conflict, trivial to resolve.

---

4. Story syntax

Current (Ch 5, still valid):

```
[MINIGAME "RECONSTRUCT — VOICE MEMO" 30s → success: ch5_fragment_full / failure: ch5_fragment_parti
```

Extended (adds the game id and an optional quality flag):

```
[MINIGAME ch6_marker_watch "MARKER DRIFT" 45s → success: ch6_markers_read / failure: ch6_markers_m
[MINIGAME ch2_frequency_trace "TRACE — RESIDUAL SIGNAL" 30s quality: frequency_read → success: ch2_
```

The engine flag gains `id`, `app`, and `quality_flag`; everything else is unchanged. Old syntax keeps working (defaults to `app: terminal`, the existing grid game).

---

5. Building with zero dependencies (Arinze)

Until the real shells exist, mount any game in a scratch harness:

```
// src/screens/minigames/__harness.jsx — dev only, never shipped
const [t, setT] = useState(45);
useEffect(() => { const i = setInterval(() => setT(x => x-1), 1000); return () => clearInterval

<MarkerWatch
  config={{ id: 'ch6_marker_watch', label: 'MARKER DRIFT', timeLimitSeconds: 45, params: { mark
  timeLeft={t}
  onResolve={r => console.log('RESOLVED', r)}
/>
```

Point a temporary route or the desk at it. When Luca's shells land, delete the harness — the games need no changes, because the props are identical.

6. Idle state

When `flags.__minigame__` is null, each app renders its idle screen. Blank, plus one dormant line so it reads as “working, nothing to do” rather than broken:

- **Terminal** — blinking cursor on an empty prompt
- **Lens** — `NO ACTIVE READ`
- **Feeds** — `NO LIVE FEED`

Owned by the host, not the games.